Name: A.Midhuna
Register No: 192521302

ASSIGNMENT 3 – CO3

CO3 Coverage: Construct C++ programs using constructors, destructors and operator overloading to control object creation, destruction and operator behaviour – default, parameterised, overloaded, copy and dynamic constructors, destructors, defining and overloading operators, and the rules for operator overloading.

Nature of Assignment: Individual

Duration: 1 Week

Total Marks: 50

Submission Date: _____

Bloom's Levels Assessed: K5 – Evaluate (every task in this assignment is at K5

level) Mandatory Execution and GitHub Submission

• Every student must individually execute every task in this assignment, including all "Complete the Code" programs.

• Create a GitHub repository named DSA01-<RegisterNumber>-Assignment3 and share it with the course faculty (public, or private with the faculty added as a collaborator).

• Keep one folder for each task (for example: Task01, Task02 … or Q1, Q2 …), containing the C++ source files, input files and output screenshots.

• Include a README.md with your name, register number, compiler and version (for example g++ 13), the exact g++ command used to compile (for example g++ -std=c++17 -Wall -Wextra), the steps to run each program, and a short note of the result of each task.

• Commit your work progressively – at least one commit for each task. The commit history will be checked; a single upload of all files on the last day will not be accepted.

• Submit the GitHub link below and on the cover page of your report. An assignment submitted without a working GitHub link will not be evaluated.

GitHub Repository Link:

_____ How to Answer the

Additional Questions

Complete the Code: Copy the given code into your repository, replace every TODO with

working code without changing the rest of the program, execute it and compare your output with the expected output.

Predict the Output: Write the exact output of the given program before running it. Then run it and explain every difference between your prediction and the actual output, using the C++ rules involved. Predictions written after running the program will not be credited.
Alternative Concept and Complexity Analysis: Describe how the solution would change if the alternative concept were used, with code or pseudocode, and analyse and compare the time complexity and space complexity of both solutions using Big-O notation, supported by your own measurements wherever possible.

Assignment Principle

Every question asks you to design, implement, analyse and evaluate the solution yourself, so that you demonstrate evidence of:

Design → Implementation → Analysis → Evaluation

Q1 – Constructors and Constructor Overloading 6 Marks

Bloom's Level: K5 – Evaluate

Problem Context – Library Book Records

A library creates Book objects in several situations: a new book with full details, a donated book with only a title, and an empty record that will be filled later. Some records were created with garbage values in the number of copies.

Implementation Tasks

You must:

• Write a default constructor, a parameterised constructor and overloaded constructors for the three situations

• Use a member initializer list in every constructor

• Use constructor delegation to avoid repeating code

• Show that no Book can be created with an invalid number of copies

Design & Analysis

You must:

• Explain when the compiler supplies a default constructor and when it does not

• Explain why member initializer lists are preferred to assignment inside the constructor body

• Explain the order in which members are initialised (declaration order, not list order) •

Draw the state of three objects after construction

Experimental Investigation

You must:

• Create Book objects with every constructor and print them

• Remove the default constructor and record the compiler error when an array of Books is created

• Demonstrate a member initialised in the wrong order and the -Wreorder warning •

Test a negative number of copies

Design Question

When should a constructor be marked explicit? Show a call that compiles without explicit and fails with it.

Alternative Concept and Complexity Analysis

If one constructor with default arguments (Book(std::string t = "", int copies = 0, …)) were used instead of several overloaded constructors and delegation, what would the solution be? Analyse the time and space of object creation (O(1) in both) and compare which design checks the number of copies in fewer places.

ANSWER:

Constructors and Constructor Overloading

1. C++ Program

A library needs three types of Book objects:

- Full book details.
- Donated book with only a title.
- Empty book record to be filled later.

We use constructor overloading, member initializer lists and constructor delegation to create these objects safely.

```
#include <iostream>

#include <string>
```

```
#include <stdexcept>
using namespace std;
class Book {
private:
    string title;
    string author;
    int copies;
public:
    // Default constructor
    Book() : Book("", "", 0) {}
// Title-only constructor
    Book(string t) : Book(t, "Unknown", 1) {}
// Full parameterised constructor
    Book(string t, string a, int c)
        : title(t), author(a), copies(c)
    {
        if (c < 0)
            throw invalid_argument("Copies cannot be negative");
    }
void display() const {
        cout << "Title: " << title
             << ", Author: " << author
             << ", Copies: " << copies << endl;
    }
};
int main() {
    try {
```

```
        Book b1;
        Book b2("C++ Programming");
        Book b3("Java Basics", "James Gosling", 5);
        b1.display();
        b2.display();
        b3.display();
 // Negative copies test
        Book b4("Python", "Guido", -3);
        b4.display();
    }
    catch (const invalid_argument& e) {
        cout << "Error: " << e.what() << endl;
    }
 return 0;
}
```

2. Expected Output

Title: , Author: , Copies: 0

Title: C++ Programming, Author: Unknown, Copies: 1

Title: Java Basics, Author: James Gosling, Copies: 5

Error: Copies cannot be negative

Explanation:

- b1 creates an empty book.
- b2 creates a donated book with only a title.
- b3 creates a book with complete details.
- b4 is rejected because the number of copies is negative.

3. Default Constructor and Member Initializer List

Default constructor:

- If no constructor is declared, the compiler normally provides an implicit default constructor.
- If we declare a parameterised constructor, the compiler does not automatically provide a default constructor.
- We must write the default constructor ourselves if required.

Member initializer list:

Example:

```
Book(string t, string a, int c)
   : title(t), author(a), copies(c) {}
```

Advantages:

1. Initializes members directly.
2. Avoids unnecessary assignment.
3. Required for const and reference members.
4. Makes constructor code efficient and clear.

4. Order of Member Initialization

C++ initializes data members in their declaration order, not according to the order written in the initializer list.

Example:

```
class Test {
   int x;
   int y;
public:
   Test() : y(20), x(10) {}
};
```

Here, x initializes first because it is declared first, even though y appears first in the initializer list.

5. State of Three Objects After Construction

Object	Title	Author	Copies
b1	Empty	Empty	0

b2	C++ Programming	Unknown	1
b3	Java Basics	James Gosling	5

6. Experimental Investigation

A. Remove the Default Constructor

If the default constructor is removed:

Book books[3];

Compiler error: No matching default constructor is available to create the array elements.

Reason: Every array element requires a default constructor.

B. Demonstrate -Wreorder Warning

```
class Test {
   int x;
   int y;
public:
   Test() : y(20), x(10) {}
};
```

Compile using:

g++ program.cpp -Wall -Wextra -Wreorder

Expected warning: y will be initialized after x, even though y appears first in the initializer list.

C. Test Negative Copies

Book b("Python", "Guido", -3);

Output:

Error: Copies cannot be negative

The exception prevents the invalid Book object from being successfully created.

7. When Should a Constructor Be Explicit?

The explicit keyword prevents unwanted automatic type conversion.

Without explicit:

```
class Book {
public:
    Book(string title) {}
};
void displayBook(Book b) {}
int main() {
    displayBook("C++ Programming");
}
```

This compiles because the string is automatically converted into a Book object.

With explicit:

```
class Book {
public:
    explicit Book(string title) {}
};
```

Now:

```
displayBook("C++ Programming");
```

Produces a compiler error because automatic conversion is not allowed.

Correct call:

```
displayBook(Book("C++ Programming"));
```

8. Alternative Concept – Default Arguments

Instead of multiple overloaded constructors, we can use one constructor with default arguments.

```
Book(string t = "", string a = "Unknown", int c = 0)
    : title(t), author(a), copies(c)
{
    if (c < 0)
        throw invalid_argument("Invalid copies");
}
```

Objects can be created as:

Book b1;

Book b2("C++ Programming");

Book b3("Java Basics", "James", 5);

Comparison:

Constructor overloading	Default arguments
Multiple constructors	Single constructor
Uses constructor delegation	Uses default values
Clear separate situations	Shorter code
Validation can be centralised	Validation in one place

9. Time and Space Complexity

Operation	Time	Space
Default object creation	O(1)	O(1)
Title-only creation	O(1)	O(1)
Full object creation	O(1)	O(1)

Each Book object contains a fixed number of data members. Therefore, object creation takes constant time and space, excluding variable-length string storage and allocation costs.

10. Final Evaluation

Constructor overloading is suitable when a library creates books in different situations. Constructor delegation avoids repeating initialization code, and member initializer lists improve efficiency.

The negative-copy validation ensures that invalid Book objects cannot be created successfully.

Conclusion: I prefer constructor overloading with constructor delegation because it provides clear object creation methods, maintains data safety and reduces code duplication. Default arguments are also suitable for smaller programs, but overloaded constructors make different creation situations easier to understand.

Q2 – Copy Constructors: Shallow and Deep Copies 7 Marks

Bloom's Level: K5 – Evaluate

Problem Context – Hospital Prescription Records

A Prescription object holds a dynamically allocated array of medicine names. When a doctor copies a prescription to modify it for a follow-up visit, the original prescription also changes, and the program sometimes crashes when it ends.

Implementation Tasks

You must:

• Implement the class first without a copy constructor and observe the problem

• Implement a deep-copying copy constructor, a copy assignment operator and a destructor

• Verify that copies are independent

• Run the program under -fsanitize=address (or valgrind) for both versions

Design & AnalysisYou must:

• Draw memory diagrams for a shallow copy and a deep copy

• Explain why the shallow version crashes at the end (double delete) • State

the Rule of Three and explain why the three functions go together • Explain

the three situations in which the copy constructor is called

Experimental Investigation

You must:

• Add a message in the copy constructor and count how many copies are made when a Prescription is passed by value and by const reference

• Measure the time to copy prescriptions of 10, 1,000 and 100,000 medicines •

Report the sanitizer output for both versions

• Test self-assignment (p = p)

Conclusion

When can you rely on the compiler-generated copy constructor, and when must you write your own?

Complete the Code

Complete TODO 1 to TODO 3 below so that every copy is independent, and execute the

program under -fsanitize=address.

```
#include <iostream>

#include <string>


class Prescription {

 int n;

 std::string* medicines; // dynamically allocated array

public:

 explicit Prescription(int count) : n(count), medicines(new std::string[count]) {}
 // TODO 1: Copy constructor: allocate a NEW array of the same size and copy every

// medicine (deep copy)

  // TODO 2: Copy assignment: guard against self-assignment, build a new array, copy,  //
release the old array and return *this
   ~Prescription()

 {

 // TODO 3: Release the dynamically allocated array

   }

void set(int i, const std::string& m) { medicines[i] = m; }

 std::string get(int i) const { return medicines[i]; }

};

int main()

{

 Prescription p(2);

 p.set(0, "Paracetamol");

 p.set(1, "ORS");

 Prescription q = p; // copy constructor

 q.set(1, "Cetirizine");

 Prescription r(1);

 r = p; // copy assignment

 p.set(0, "Ibuprofen");
```

```
 std::cout << p.get(0) << ' ' << p.get(1) << " | " << q.get(0) << ' ' << q.get(1)
<< " | " << r.get(0) << ' ' << r.get(1) << '\n';
```

} Expected output:

Ibuprofen ORS | Paracetamol Cetirizine | Paracetamol ORS

Delete your copy constructor, run the program again under the sanitizer and record the error reported.

Alternative Concept and Complexity Analysis

If the medicine names were stored in a std::vector<std::string> instead of a dynamically allocated array, the compiler-generated copy constructor would already make a deep copy (the Rule of Zero). What would the solution be? Analyse the time O(n) and space O(n) to copy a prescription of n medicines in both designs, and compare them with a shallow copy (O(1)).

ANSWER:

Copy Constructors: Shallow and Deep Copies

1. Introduction

In a hospital prescription system, a Prescription object stores medicine names using a dynamically allocated array.

When one prescription is copied, changes made to the copied object should not affect the original object.

- Shallow copy: Copies the memory address. Both objects share the same array.
- Deep copy: Creates a new array and copies every medicine separately.

Deep copy is required to avoid shared data, unexpected modifications and double deletion.

2. Completed C++ Program (TODO 1–3)

```
#include <iostream>

#include <string>

using namespace std;


class Prescription {
```

```
    int n;
    string* medicines;

public:
    explicit Prescription(int count)
        : n(count), medicines(new string[count]) {}

    // TODO 1: Deep copy constructor
    Prescription(const Prescription& other)
        : n(other.n), medicines(new string[other.n])
    {
        for (int i = 0; i < n; i++) {
            medicines[i] = other.medicines[i];
        }
        cout << "Copy constructor called\n";
    }

    // TODO 2: Copy assignment operator
    Prescription& operator=(const Prescription& other)
    {
        if (this != &other) {
            string* newMedicines = new string[other.n];

            for (int i = 0; i < other.n; i++) {
                newMedicines[i] = other.medicines[i];
            }
```

```
            delete[] medicines;

            medicines = newMedicines;
            n = other.n;
        }

        return *this;
    }

    // TODO 3: Destructor
    ~Prescription()
    {
        delete[] medicines;
    }

    void set(int i, const string& m) {
        medicines[i] = m;
    }

    string get(int i) const {
        return medicines[i];
    }
};

int main()
{
    Prescription p(2);
```

```
    p.set(0, "Paracetamol");
    p.set(1, "ORS");

    Prescription q = p; // Copy constructor

    q.set(1, "Cetirizine");

    Prescription r(1);
    r = p; // Copy assignment

    p.set(0, "Ibuprofen");

    cout << p.get(0) << ' ' << p.get(1)
         << " | " << q.get(0) << ' ' << q.get(1)
         << " | " << r.get(0) << ' ' << r.get(1)
         << '\n';

    return 0;
}
```

3. Expected Output

Copy constructor called

Ibuprofen ORS | Paracetamol Cetirizine | Paracetamol ORS

Explanation:

- p stores Paracetamol and ORS.
- q is an independent copy of p.
- Changing q to Cetirizine does not affect p.

- r receives a deep copy through copy assignment.
- Changing p to Ibuprofen does not affect q or r.

Therefore, all three objects maintain independent medicine arrays.

4. Memory Diagrams

A. Shallow Copy

In shallow copy, both objects point to the same dynamically allocated array.

Shallow copy

Problem: Both objects share the same memory.

If q changes ORS to Cetirizine, p also sees the change. When both objects are destroyed, they attempt to delete the same memory, causing a double-free error.

B. Deep Copy

In deep copy, each object gets its own separate array.

Deep copy

Independent memory

Changing one prescription does not affect the other.

5. Rule of Three

The Rule of Three states that if a class requires one of these three special functions, it generally requires all three:

1. Copy constructor: Creates a new object by copying another object.
2. Copy assignment operator: Copies data into an already existing object.
3. Destructor: Releases dynamically allocated memory.

These three functions work together to manage memory safely.

6. Three Situations Where Copy Constructor Is Called

1. Initializing an object from another object

```
Prescription q = p;
```

2. Passing an object by value

```
void display(Prescription p) {
    // Function body
```

}

A copy is created for the function parameter.

3. Returning an object by value

```
Prescription createPrescription() {
    Prescription p(2);
    return p;
}
```

A copy may be needed, but modern C++ can eliminate it through copy elision.

Pass by Value vs Pass by Const Reference

```
void test1(Prescription p) { }
void test2(const Prescription& p) { }
```

Method	Copy constructor
Pass by value	Usually called
Pass by const reference	Not called

Passing by const reference avoids unnecessary copying.

7. Self-Assignment Test

```
p = p;
```

The condition:

```
if (this != &other)
```

checks whether the source and destination are the same object.

If they are the same, copying is skipped. This prevents the object from deleting its own data before copying it.

8. Sanitizer Investigation

Compile the deep-copy program using:

```
g++ prescription.cpp -o prescription -fsanitize=address -g
./prescription
```

Deep copy – Expected result

- Program executes successfully.

- No double-free error.
- No memory leak is expected.
- Each object deletes its own array.

Without a copy constructor:

The compiler-generated copy constructor copies only the pointer address. When the objects are destroyed, the same memory is deleted multiple times.

Expected AddressSanitizer diagnostic:

ERROR: AddressSanitizer: attempting double-free

The exact diagnostic and stack trace depend on the compiler and system.

9. Experimental Investigation – Copy Timing

We need to measure copying prescriptions containing 10, 1,000 and 100,000 medicines.

Use this benchmark function with the deep-copy class. Remove the copy-constructor print message during timing so that console output does not affect the measurements.

```
#include <chrono>

using namespace std::chrono;


void benchmark(const Prescription& p, int repetitions)

{

   auto start = high_resolution_clock::now();


   size_t checksum = 0;


   for (int i = 0; i < repetitions; i++) {

      Prescription copy(p);

      checksum += copy.get(0).size();

   }


   auto end = high_resolution_clock::now();
```

```
    auto duration =
        duration_cast<microseconds>(end - start).count();


    std::cout << "Time: " << duration << " microseconds\n";
}
```

Use these test sizes:

Medicines	Repetitions
10	100000
1,000	1,000
100,000	10

Record the actual execution time.

Medicines	Execution time
10	Record your result
1,000	Record your result
100,000	Record your result

Expected observation: As the number of medicines increases, deep-copying takes more time because every medicine name must be copied.

10. Alternative Concept – Using std::vector

Instead of dynamically allocating an array manually, we can use std::vector<std::string>.

Example:

```
#include <iostream>
#include <string>
#include <vector>
using namespace std;


class Prescription {
private:
```

```
    vector<string> medicines;

public:
    explicit Prescription(int n) : medicines(n) {}

    void set(int i, string m) {
        medicines[i] = m;
    }

    string get(int i) const {
        return medicines[i];
    }
};

int main() {
    Prescription p(2);

    p.set(0, "Paracetamol");
    p.set(1, "ORS");

    Prescription q = p;

    q.set(1, "Cetirizine");

    cout << p.get(1) << endl;
    cout << q.get(1) << endl;
}
```

Output:

ORS

Cetirizine

The vector automatically manages memory and performs independent copying of its elements.

This follows the Rule of Zero, meaning we do not need to write a copy constructor, copy assignment operator or destructor ourselves.

11. Time and Space Complexity

Copy method	Time complexity	Extra space
Shallow copy	O(1)	O(1)
Deep copy using array	O(n)	O(n)
Deep copy using vector	O(n)	O(n)

Here, n represents the number of medicines.

- Shallow copy is faster but unsafe because both objects share memory.
- Deep copy using arrays creates independent memory but requires manual memory management.
- Vector provides automatic memory management and safe copying.

12. Final Conclusion

Deep copying is necessary when a class contains dynamically allocated memory and each object must maintain independent data.

The Rule of Three helps manage memory safely through the copy constructor, copy assignment operator and destructor.

I prefer std::vector because it reduces manual memory management and avoids common errors such as double deletion and memory leaks. However, manually implemented deep copying is useful for understanding dynamic memory allocation and copy operations.

Q3 – Dynamic Constructors and Destructors 7 Marks

Bloom's Level: K5 – Evaluate

Problem Context – Auditorium Seat Booking

An auditorium's seat map is created when an event is set up; the number of rows and seats

per row differs from event to event. When the event object is destroyed, all memory must be released and a closing summary written.

Implementation Tasks

You must:

• Write a dynamic constructor that allocates a two-dimensional seat map with new • Write a destructor that releases every allocation and prints a summary • Create events on the stack, on the heap (new/delete) and inside a block • Book and cancel seats

Design & Analysis

You must:

• Explain the order in which constructors and destructors run for local, block-scope and dynamically created objects
• Explain what happens if delete is forgotten, and if delete is used instead of delete[] • Explain why the destructor takes no arguments and cannot be overloaded • Draw the memory layout of the seat map

Experimental Investigation

You must:

• Print messages in the constructor and destructor and record the order for each kind of object

• Run with -fsanitize=address, first with a missing delete and then with the correct code • Measure the memory allocated for 10 × 20, 100 × 200 and 1,000 × 2,000 seat maps • Test an event with zero rows

Conclusion

How does tying memory release to the destructor (the RAII idea) make the program safer? Predict the Output

Write the exact output of the program below BEFORE running it. Then compile and run it, and explain every difference between your prediction and the actual output.

```
#include <iostream>
#include <string>

struct Logger {
```

```
 std::string n;
 Logger(const std::string& s) : n(s) { std::cout << '+' << n << ' '; }
 Logger(const Logger& o) : n(o.n + "c") { std::cout << "copy" << n << ' '; }
~Logger() { std::cout << '-' << n << ' '; }
};

void use(Logger) { std::cout << "use "; }

int main()
{
 Logger a("A");
 {
 Logger b("B");
 }
 use(a);
 Logger* p = new Logger("P");
 delete p;
 std::cout << "end ";
}
```

My predicted output (write before running):

Explain when each object was created and destroyed, and why the copy was made and destroyed around use().

Alternative Concept and Complexity Analysis

If the seat map were one block of rows × seats elements (index r × seats + s) or a std::vector<std::vector<char>>, instead of an array of row pointers each allocated with new, what would the solution be? Analyse the access time (O(1) in all), the number of allocations (1 versus rows + 1) and the extra space for the row pointers.

ANSWER:

Dynamic Constructors and Destructors

1. Introduction

In an auditorium booking system, the number of rows and seats changes for each event. Therefore, dynamic memory allocation is used to create a two-dimensional seat map.

- Dynamic constructor: Allocates memory using new.
- Destructor: Releases allocated memory using delete[].
- Booking: Changes an available seat (0) to booked (1).
- Cancellation: Changes a booked seat (1) to available (0).

2. Completed C++ Program

This program uses a dynamic constructor, destructor, seat booking and cancellation. It also creates objects on the stack, heap and inside a block.

```
#include <iostream>
#include <string>
using namespace std;

class Auditorium {
private:
    int rows, seats;
    int** seatMap;
    string eventName;

public:
    // Dynamic constructor
    Auditorium(string name, int r, int s)
        : rows(r), seats(s), seatMap(nullptr),
          eventName(name)
    {
        cout << "Constructor: " << eventName << endl;
```

```
    if (rows < 0 || seats < 0)
        throw invalid_argument("Invalid dimensions");

    if (rows > 0 && seats > 0) {
        seatMap = new int*[rows];

        int i = 0;
        try {
            for (; i < rows; i++)
                seatMap[i] = new int[seats]{};
        }
        catch (...) {
            while (i > 0)
                delete[] seatMap[--i];

            delete[] seatMap;
            seatMap = nullptr;
            throw;
        }
    }
}

// Book a seat
void bookSeat(int r, int s) {
    if (r >= 0 && r < rows &&
        s >= 0 && s < seats) {
```

```
        seatMap[r][s] = 1;
        cout << "Seat booked: "
             << r << "," << s << endl;
    }
}

// Cancel a seat
void cancelSeat(int r, int s) {
    if (r >= 0 && r < rows &&
        s >= 0 && s < seats) {
        seatMap[r][s] = 0;
        cout << "Seat cancelled: "
             << r << "," << s << endl;
    }
}

// Display seat map
void display() const {
    for (int i = 0; i < rows; i++) {
        for (int j = 0; j < seats; j++)
            cout << seatMap[i][j] << " ";
        cout << endl;
    }
}

// Destructor
~Auditorium() {
```

```
        if (seatMap != nullptr) {
            for (int i = 0; i < rows; i++)
                delete[] seatMap[i];

            delete[] seatMap;
        }

        cout << "Closing event: " << eventName
             << endl;
        cout << "Total seats: " << rows * seats
             << endl;
    }
};

int main() {
    // Stack object
    Auditorium a("College Day", 2, 3);
    a.bookSeat(0, 1);
    a.cancelSeat(0, 1);
    a.display();

    // Block-scope object
    {
        Auditorium b("Cultural Event", 3, 4);
        b.bookSeat(1, 2);
    }
```

```
    // Heap object
    Auditorium* p =
        new Auditorium("Music Show", 2, 2);

    p->bookSeat(0, 0);
    delete p;

    // Zero-row event
    Auditorium c("Small Event", 0, 5);

    return 0;
}
```

3. Explanation of Constructor and Destructor Order

Object type	Constructor execution	Destructor execution
Local/stack object	When created	When scope ends
Block-scope object	When block starts creating it	When block ends
Heap object	When new is executed	When delete is executed

Important rules:

1. Constructors run when objects are created.
2. Destructors run when objects are destroyed.
3. Local objects are destroyed in reverse order of construction.
4. Heap objects remain alive until explicitly deleted or otherwise managed.

4. Memory Layout of the Seat Map

For a 2 × 3 seat map:

Dynamic memory representation

seatMap – row pointer array

Row 0 pointer

Row 1 pointer

Each row is allocated separately using new int[seats].

The seat map uses:

- One allocation for the row pointers.
- One allocation for each row.
- Total allocations = rows + 1.

5. Destructor and Memory Management

Why is a destructor important?

The destructor releases dynamically allocated memory when an object is destroyed.

```
~Auditorium() {
    for (int i = 0; i < rows; i++)
        delete[] seatMap[i];

    delete[] seatMap;
}
```

What happens if delete is forgotten?

- Dynamically allocated memory is not released.
- Memory leaks occur.
- Repeated event creation can increase memory consumption.

What happens if delete is used instead of delete[]?

- delete is meant for a single object.
- delete[] is required for arrays.
- Using the wrong form causes undefined behaviour and may result in memory errors.

Why can't a destructor take arguments or be overloaded?

- A destructor takes no arguments.
- A class can have only one destructor.
- It is automatically called when an object is destroyed.

- It cannot return a value.

6. Experimental Investigation

A. Constructor and Destructor Messages

Expected order for the Logger-style object lifecycle:

- Stack object: constructor first, destructor at the end of its scope.
- Block object: constructor when entering its lifetime, destructor when leaving the block.
- Heap object: constructor during new, destructor during delete.

B. AddressSanitizer Test

Compile using:

g++ auditorium.cpp -o auditorium -fsanitize=address -g

./auditorium

Test 1 – Missing delete

If the heap object is not deleted:

```
Auditorium* p =
    new Auditorium("Music Show", 2, 2);


// delete p;  // Removed intentionally
```

Expected sanitizer result:

ERROR: LeakSanitizer: detected memory leaks

The heap object and its seat-map allocations remain unreleased.

Test 2 – Correct delete

```
delete p;
```

Expected result:

- Destructor executes.
- All row arrays are released.
- Row pointer array is released.
- No memory leak is expected.

C. Memory Measurement

Assume a 64-bit system where an integer uses 4 bytes and a pointer uses 8 bytes.

Memory formula:

\[M=(rows \times seats \times 4)+(rows \times 8) \]

Seat map	Seat memory	Pointer memory	Total
10 × 20	800 B	80 B	880 B
100 × 200	80,000 B	800 B	80,800 B
1,000 × 2,000	8,000,000 B	8,000 B	8,008,000 B

These are theoretical payload sizes, excluding allocator metadata and alignment overhead.

D. Zero Rows Test

Auditorium a("Small Event", 0, 5);

Expected behaviour:

- No seat-map memory is allocated.
- No booking is possible.
- Destructor executes safely.
- Total seats = 0.

The program should also reject negative dimensions.

7. Predict the Output – Logger Program

My predicted output (before execution):

+A +B -B copyAc use -Ac +P -P end -A

Explanation of each step:

Step	Operation	Output
1	Create object a	+A
2	Create object b	+B
3	Block ends; destroy b	-B
4	Pass a by value	copyAc

5	Execute function body	use
6	Destroy copied parameter	-Ac
7	Create heap object p	+P
8	Delete heap object	-P
9	Print end	end
10	Destroy local object a	-A

Why is a copy created around use()?

The function is declared as:

void use(Logger);

It accepts the Logger object by value. Therefore, a copy of a is created for the function parameter. After the function finishes, that copied object is destroyed automatically.

8. Alternative Design and Complexity Analysis

Instead of allocating every row separately, we can use a single continuous memory block or std::vector.

Method 1: Single Continuous Memory Block

```
int* seatMap = new int[rows * seats]{};

// Access seat

seatMap[r * seats + s] = 1;

// Release memory

delete[] seatMap;
```

Advantages:

- Only one dynamic memory allocation.
- Better memory locality.
- Less pointer overhead.
- O(1) seat access.

Method 2: Using Vector

```
vector<vector<int>> seatMap(

   rows, vector<int>(seats, 0)
```

);

Advantages:

- Automatic memory management.
- No need to write a destructor for the vector.
- Easy booking and cancellation.
- Follows the Rule of Zero.

Comparison

Feature	2D dynamic array	Single block	Vector
Allocations	rows + 1	1	Usually rows + 1
Access time	O(1)	O(1)	O(1)
Memory management	Manual	Manual	Automatic
Extra row pointers	Yes	No	Managed internally
Safety	Requires care	Requires care	Safer

9. Final Conclusion – RAII

RAII means Resource Acquisition Is Initialization.

It connects resource management with object lifetime.

- Constructor allocates the required memory.
- Destructor automatically releases the memory.
- It reduces memory leaks.
- It makes the program safer and easier to maintain.
- It ensures resources are released when objects leave their scope.

Final decision: I prefer std::vector for a real auditorium booking system because it automatically manages memory. However, dynamic constructors and destructors help us understand how memory allocation and deallocation work in C++.

Q4 – Overloading Arithmetic and Stream Operators 7 Marks

Bloom's Level: K5 – Evaluate

Problem Context – Money Arithmetic for a Billing Counter

A billing counter adds, subtracts and compares amounts of money stored in rupees and paise. The present code calls functions such as add(a, b) and print(c), which makes the bill calculation hard to read.

Implementation Tasks

You must:

• Write a Money class storing paise in a long long

• Overload +, -, +=, == and < as appropriate

• Overload << (and >>) for printing and reading amounts

• Write the bill calculation using the overloaded operators

Design & Analysis

You must:

• Explain why + returns a new object while += returns a reference to *this • Decide which operators are members and which are friends, and justify each • Explain why << must be a non-member function

• Show how a + b is translated by the compiler into a function call

Experimental Investigation

You must:

• Test addition that carries paise into rupees (Rs 0.75 + Rs 0.50) • Test subtraction that produces a negative amount and decide how to handle it • Compare the readability of the bill code with and without operator overloading • Compare the result with a double-based version for 1,000,000 additions

Engineering Decision

Should Money allow multiplication by another Money? Decide which operators make sense for money and which do not.

Complete the Code

Complete TODO 1 and TODO 2 below, and execute the program.

```
#include <iomanip>
#include <iostream>
class Money {
 long long paise;
public:
```

```
 Money(long long rupees = 0, int p = 0) : paise(rupees * 100 + p) {}
 // TODO 1: Overload + as a member function that returns a new Money object

 // TODO 2: Overload == as a member, and << as a friend that prints "Rs 200.00"
};

int main()
{
 Money a(120, 75), b(79, 25);
 Money c = a + b;
 std::cout << c << '\n' << std::boolalpha << (c == Money(200)) << '\n'; }
```

Expected output:

Rs 200.00

true

Add a test where paise carry into rupees (Rs 0.75 + Rs 0.50).

Alternative Concept and Complexity Analysis

If the operators were written as non-member friend functions instead of member functions (or replaced by named functions add() and equals()), what would the solution be? Analyse whether time or space changes (all are O(1)), and explain which form allows 100 + m as well as m + 100.

ANSWER:

Overloading Arithmetic and Stream Operators –

1. Introduction

In a billing counter, money is stored in rupees and paise. Operator overloading makes calculations easier and improves code readability.

We create a Money class using long long to store the amount in paise.

Operators used:

- + – Addition
- - – Subtraction
- += – Add and update
- == – Check equality
- < – Compare amounts
- << – Display money
- >> – Read money

2. Completed C++ Program (TODO 1 and TODO 2)

```
#include <iomanip>
#include <iostream>
#include <sstream>
using namespace std;
class Money {
    long long paise;
public:
    Money(long long rupees = 0, int p = 0)
        : paise(rupees * 100 + p) {}
  // TODO 1: Addition operator
    Money operator+(const Money& other) const {
        return Money(0, static_cast<int>(0));
    }
// Subtraction
    Money operator-(const Money& other) const {
        return Money(0, 0) + Money();
    }
 // += operator
```

```
    Money& operator+=(const Money& other) {
        paise += other.paise;
        return *this;
    }
 // TODO 2: Equality operator
    bool operator==(const Money& other) const {
        return paise == other.paise;
    }
// Less-than operator
    bool operator<(const Money& other) const {
        return paise < other.paise;
    }

    // Stream output
    friend ostream& operator<<(ostream& out,
                               const Money& m) {
        long long value = m.paise;
        if (value < 0) {
            out << "-Rs " << (-value / 100) << "."
                << setw(2) << setfill('0')
                << (-value % 100);
        } else {
            out << "Rs " << value / 100 << "."
                << setw(2) << setfill('0')
                << value % 100;
        }
        out << setfill(' ');
```

```
        return out;
    }

    // Stream input
    friend istream& operator>>(istream& in, Money& m) {
        long long rupees;
        int p;
        if (in >> rupees >> p) {
            m.paise = rupees * 100 + p;
        }
        return in;
    }
};

int main() {
    Money a(120, 75), b(79, 25);

    Money c = a + b;

    cout << c << '\n';
    cout << boolalpha << (c == Money(200)) << '\n';

    Money x(0, 75), y(0, 50);
    cout << x + y << '\n';

    return 0;
}
```

Important correction: The two placeholder arithmetic implementations above must be replaced with the following correct implementations before compiling:

```
Money operator+(const Money& other) const {
    Money result;
    result.paise = paise + other.paise;
    return result;
}

Money operator-(const Money& other) const {
    Money result;
    result.paise = paise - other.paise;
    return result;
}
```

3. Corrected Complete Program

Use this final version for execution. It includes all required operators and correct arithmetic implementations.

```
#include <iostream>
#include <iomanip>
using namespace std;
class Money {
    long long paise;
public:
    Money(long long rupees = 0, int p = 0)
        : paise(rupees * 100 + p) {}
   // Addition
    Money operator+(const Money& other) const {
        Money result;
        result.paise = paise + other.paise;
```

```
    return result;
}

// Subtraction
Money operator-(const Money& other) const {
    Money result;
    result.paise = paise - other.paise;
    return result;
}

// Addition assignment
Money& operator+=(const Money& other) {
    paise += other.paise;
    return *this;
}

// Equality
bool operator==(const Money& other) const {
    return paise == other.paise;
}

// Less than
bool operator<(const Money& other) const {
    return paise < other.paise;
}

// Output operator
```

```
    friend ostream& operator<<(ostream& out,
                               const Money& m) {
        long long v = m.paise;
        if (v < 0) {
            out << "-Rs " << -v / 100 << "."
                << setw(2) << setfill('0') << -v % 100;
        } else {
            out << "Rs " << v / 100 << "."
                << setw(2) << setfill('0') << v % 100;
        }
        out << setfill(' ');
        return out;
    }

    // Input operator: enter rupees and paise separately
    friend istream& operator>>(istream& in, Money& m) {
        long long r;
        int p;

        if (in >> r >> p)
            m.paise = r * 100 + p;

        return in;
    }
};

int main() {
```

```
    Money a(120, 75), b(79, 25);

    Money c = a + b;

    cout << c << '\n';
    cout << boolalpha << (c == Money(200)) << '\n';

    // Paise carry test
    Money x(0, 75), y(0, 50);
    cout << x + y << '\n';

    // Subtraction test
    Money d(10, 25), e(15, 50);
    cout << d - e << '\n';

    // += test
    a += b;
    cout << a << '\n';

    // Comparison test
    cout << (d < e) << '\n';

    return 0;
}
```

4. Expected Output

```
Rs 200.00
true
```

Rs 1.25

-Rs 5.25

Rs 200.00

true

Explanation:

- Rs 120.75 + Rs 79.25 = Rs 200.00.
- Equality returns true because both amounts are equal.
- Rs 0.75 + Rs 0.50 = Rs 1.25 (paise carry into rupees).
- Rs 10.25 − Rs 15.50 = −Rs 5.25.
- += updates the existing object.
- < checks whether the first amount is smaller.

5. Why Does + Return a New Object and += Return a Reference?

Addition (+):

Money c = a + b;

- Creates a new Money object.
- Does not change the original objects.
- Returns the calculated result.

Addition assignment (+=):

a += b;

- Updates the existing object a.
- Returns a reference to *this.
- Avoids creating an unnecessary result object.

6. Member Functions and Friend Functions

Operator	**Design**	**Reason**
+	Member	Uses the current Money object
-	Member	Performs subtraction
+=	Member	Updates the current object

==	Member	Compares two amounts
<	Member	Compares two amounts
<<	Friend	Left operand is cout
>>	Friend	Left operand is cin

Why must << be a non-member?

The expression:

cout << money;

has cout as the left operand. Since cout is an ostream object, the operator cannot be a member of the Money class. Therefore, it is implemented as a non-member friend function.

7. How Does the Compiler Translate a + b?

When we write:

Money c = a + b;

The compiler treats it as:

Money c = a.operator+(b);

This calls the overloaded addition function.

8. Experimental Investigation

A. Paise Carry Test

Money a(0, 75);

Money b(0, 50);

cout << a + b;

Output:

Rs 1.25

B. Negative Subtraction Test

Money a(10, 25);

Money b(15, 50);

cout << a - b;

Output:

-Rs 5.25

Negative amounts can represent refunds, credits or outstanding balances. A billing system that prohibits negative payments should validate this separately.

C. Readability Comparison

Without operator overloading:

Money c = add(a, b);

print(c);

With operator overloading:

Money c = a + b;

cout << c;

Operator overloading makes the billing calculation shorter and easier to understand.

D. Double vs Integer Performance

Test 1,000,000 additions using:

- Version 1: double for money.
- Version 2: long long for paise.

Feature	**Double**	**Long long**
Storage	Floating point	Integer
Precision	Possible rounding errors	Exact paise arithmetic within range
Time complexity	O(1) per addition	O(1) per addition
Accuracy for currency	Less reliable	More reliable

Both perform constant-time addition. Actual execution time should be measured using chrono with the same compiler optimisation and test data.

9. Engineering Decision – Should Money Support Multiplication?

Multiplication of two Money objects is generally not meaningful.

For example:

Money a(100);

Money b(200);

Multiplying Rs 100 by Rs 200 produces a squared monetary unit, which is not a normal billing amount.

However, multiplying money by a quantity is meaningful.

Example:

Rs 100 × 5 items = Rs 500

Therefore, multiplication by an integer quantity can be supported, but multiplication of two Money objects should generally not be supported.

10. Alternative Concept – Friend Functions

Addition can also be implemented using a friend function:

```
friend Money operator+(const Money& a,
                       const Money& b) {
   Money result;
   result.paise = a.paise + b.paise;
   return result;
}
```

Another alternative is using named functions:

```
Money add(Money a, Money b);
bool equals(Money a, Money b);
```

Member vs Friend vs Named Functions

Feature	**Member**	**Friend**	**Named function**
Syntax	a + b	a + b	add(a,b)
Private access	Yes	Yes	No, unless given access
Direct readability	Good	Good	More verbose
Supports 100 + m	No	Yes, with suitable implicit conversion	No operator syntax

Important: A non-member friend operator+ allows both 100 + m and m + 100 when the Money constructor permits implicit conversion from an integer.

11. Time and Space Complexity

Operation	Time	Extra space
Addition	O(1)	O(1)
Subtraction	O(1)	O(1)
Equality	O(1)	O(1)
Comparison	O(1)	O(1)
Stream output	O(1) for fixed-size amount	O(1)

12. Final Conclusion

Operator overloading makes money calculations simple and readable. Storing amounts as integer paise avoids floating-point rounding problems. The +, -, +=, ==, <, << and >> operators provide convenient billing operations.

Final decision: I prefer integer-based Money storage with overloaded operators because it provides accurate calculations, clear syntax and better maintainability for billing applications.

Q5 – Overloading Increment, Comparison and Subscript Operators 6

Marks Bloom's Level: K5 – Evaluate

Problem Context – Music Playlist and Track Timer

A music player keeps a playlist of tracks and a Time object for the current position. The player needs time++ and ++time to advance by one second, comparisons between times, and playlist[i] to reach the i-th track with a range check.

Implementation Tasks

You must:

• Overload prefix and postfix ++ for Time

• Overload == and < for Time

• Overload [] for Playlist with a range check, in both a const and a non-const version •

Use the operators in a short playback simulation

Design & Analysis

You must:

• Explain how the compiler tells prefix from postfix ++ (the int dummy parameter) •

Explain why postfix ++ must return a copy and is slower

• Explain why operator[] needs a const and a non-const version

• Explain what should happen when an index is out of range

Experimental Investigation

You must:

• Verify the values of a and b after a = t++ and b = ++t

• Measure the time of 10,000,000 prefix and postfix increments at -O0 and -O2 •

Test playlist[-1] and playlist[size]

• Test minute and hour roll-over (59:59 → 1:00:00)

Conclusion

Why do many programmers prefer ++i to i++ for class types?
Predict the Output

Write the exact output of the program below BEFORE running it. Then compile and run it, and explain every difference between your prediction and the actual output.

```
#include <iostream>

struct Counter {

 int v;

 Counter& operator++() { ++v; return *this; } // prefix  Counter

operator++(int) { Counter old = *this; ++v; return old; } // postfix };


int main()

{

 Counter c{5};

 Counter a = c++;

 Counter b = ++c;

 std::cout << a.v << ' ' << b.v << ' ' << c.v << '\n';

}
```

My predicted output (write before running):

Explain why a and b have different values, and which version of ++ makes a copy of the object.

Alternative Concept and Complexity Analysis

If playlist access used an at(i) function that throws std::out_of_range, or an unchecked operator[] like a built-in array, instead of operator[] with a range check that prints an error, what would the solution be? Analyse the time per access (O(1) in all, with one extra comparison for checked access) and measure the cost of the check for 10,000,000 accesses.

ANSWER:

1. C++ Program
The following program demonstrates prefix and postfix increment, comparison operators, and subscript operator overloading with range checking.

```
#include <iostream>
#include <vector>
using namespace std;

class Time {
public:
   int hour, minute, second;

   Time(int h = 0, int m = 0, int s = 0) {
      hour = h;
      minute = m;
      second = s;
   }

   // Convert time into seconds
   int totalSeconds() const {
      return hour * 3600 + minute * 60 + second;
   }

   // Prefix increment
   Time& operator++() {
      ++second;
      if (second == 60) {
         second = 0;
         ++minute;
      }
      if (minute == 60) {
         minute = 0;
         ++hour;
      }
      return *this;
```

```
    }

    // Postfix increment
    Time operator++(int) {
        Time old = *this;
        ++(*this);
        return old;
    }

    // Equality operator
    bool operator==(const Time& t) const {
        return totalSeconds() == t.totalSeconds();
    }

    // Less-than operator
    bool operator<(const Time& t) const {
        return totalSeconds() < t.totalSeconds();
    }

    void display() const {
        cout << hour << ":" << minute << ":" << second;
    }
};

class Playlist {
    vector<string> tracks;

public:
    Playlist() {
        tracks = {"Song A", "Song B", "Song C"};
    }

    // Non-const subscript
    string& operator[](int index) {
        if (index < 0 || index >= (int)tracks.size()) {
            throw out_of_range("Invalid playlist index");
        }
        return tracks[index];
    }

    // Const subscript
    const string& operator[](int index) const {
        if (index < 0 || index >= (int)tracks.size()) {
            throw out_of_range("Invalid playlist index");
        }
```

```
        return tracks[index];
    }

    int size() const {
        return tracks.size();
    }
};

int main() {
    try {
        Time t(0, 0, 5);

        Time a = t++;
        Time b = ++t;

        cout << "Postfix value: ";
        a.display();

        cout << "\nPrefix value: ";
        b.display();

        cout << "\nCurrent time: ";
        t.display();

        cout << "\n\nTime comparison: ";

        if (a < b)
            cout << "a is smaller than b";

        cout << "\n\nPlaylist:\n";

        Playlist p;

        for (int i = 0; i < p.size(); i++)
            cout << p[i] << endl;

        cout << "\nAccessing invalid index:\n";
        cout << p[5];
    }
    catch (const out_of_range& e) {
        cout << e.what() << endl;
    }

    return 0;
}
```

2. Design and Analysis

1. How does the compiler distinguish prefix and postfix?

- Prefix increment uses operator++() without arguments.
- Postfix increment uses operator++(int).
- The dummy integer parameter distinguishes postfix from prefix.

2. Why does postfix return a copy?

Postfix returns the original value before incrementing. Therefore, it creates a temporary copy of the object. This requires extra memory and copying time.

3. Why are const and non-const operator[] needed?

- Non-const version allows users to access and modify playlist tracks.
- Const version allows read-only access to a constant playlist.

4. What happens when an index is out of range?

The program throws out_of_range exception instead of accessing invalid memory. The exception is handled using try-catch.

3. Experimental Investigation

A. Verify values of a and b

Given:

Time t(0, 0, 5);

Time a = t++;
Time b = ++t;

Statement	a	b	t
Initial value	–	–	5
a = t++	5	–	6
b = ++t	5	7	7

Result:

- Postfix (t++) returns the old value.
- Prefix (++t) increments first and returns the new value.

B. Performance measurement

Use the following code to measure 10,000,000 increments.

```
#include <iostream>
#include <chrono>
using namespace std;
using namespace chrono;

class Counter {
public:
   long long value = 0;

   Counter& operator++() {
```

```
        ++value;
        return *this;
    }

    Counter operator++(int) {
        Counter old = *this;
        ++value;
        return old;
    }
};

int main() {
    const int N = 10000000;
    Counter c;

    auto start = steady_clock::now();

    for (int i = 0; i < N; i++)
        ++c;

    auto end = steady_clock::now();

    cout << "Prefix time: "
         << duration<double, milli>(end-start).count()
         << " ms\n";

    c.value = 0;

    start = steady_clock::now();

    for (int i = 0; i < N; i++)
        c++;

    end = steady_clock::now();

    cout << "Postfix time: "
         << duration<double, milli>(end-start).count()
         << " ms\n";
}
```

Compile and execute:

```
g++ program.cpp -O0 -o test0
./test0

g++ program.cpp -O2 -o test2
./test2
```

Optimization	Prefix	Postfix
-O0	Measure execution time	Measure execution time
-O2	Measure execution time	Measure execution time

Actual timings depend on the processor, compiler and optimization. The compiler may optimize away unnecessary copies at -O2, so the difference may become very small.

C. Test invalid playlist indexes

Test:

```
cout << p[-1];
cout << p[p.size()];
```

Expected result:

Invalid playlist index

Both indexes are invalid:

- -1 is a negative index.
- p.size() is outside the valid range.

The exception prevents invalid access.

D. Test minute and hour rollover

```
Time t(0, 59, 59);
++t;
t.display();
```

Expected output:

1:0:0

The time changes from 00:59:59 to 01:00:00.

The increment operation resets seconds and minutes to zero and increases the hour.

4. Predict the Output

Given program:

```
Counter c{5};

Counter a = c++;
Counter b = ++c;

cout << a.v << ' ' << b.v << ' ' << c.v << '\n';
```

Correct predicted output

5 7 7

Explanation:

1. Initially, c.v = 5.
2. a = c++: Postfix stores the old value 5 in a, then increments c to 6.
3. b = ++c: Prefix increments c to 7, then stores 7 in b.
4. Finally, c.v = 7.

Therefore:

- a.v = 5
- b.v = 7
- c.v = 7

Which increment makes a copy?

Postfix c++ makes a copy of the original object before incrementing. Prefix ++c does not need this extra copy.

5. Alternative Concept and Complexity Analysis

Three approaches can be used for playlist access.

Method	Description	Time complexity
Checked operator[]	Checks index before access	O(1)
at(i)	Checks index and throws exception	O(1)
Unchecked operator[]	Direct access without checking	O(1)

Comparison:

- operator[] with range checking prints or throws an error for invalid indexes.
- at(i) automatically throws std::out_of_range when the index is invalid.
- Unchecked operator[] does not check the index and may cause undefined behaviour for invalid access.

Measuring the cost of range checking

```
#include <iostream>
#include <vector>
#include <chrono>
using namespace std;
using namespace chrono;

int main() {
   vector<int> v(100, 10);
   const int N = 10000000;
   long long sum = 0;

   auto start = steady_clock::now();

   for (int i = 0; i < N; i++)
      sum += v[i % 100];

   auto end = steady_clock::now();

   cout << "Unchecked: "
        << duration<double, milli>(end-start).count()
        << " ms\n";

   sum = 0;
   start = steady_clock::now();

   for (int i = 0; i < N; i++)
      sum += v.at(i % 100);

   end = steady_clock::now();

   cout << "Checked: "
        << duration<double, milli>(end-start).count()
```

```
        << " ms\n";
}
```
Compile with -O2 and compare the execution times. The difference may be small because both methods have O(1) access time and compilers can optimize checks in some cases.

6. Conclusion

Many programmers prefer ++i over i++ for class objects because:

- Prefix increment directly modifies the existing object.
- Postfix increment creates a temporary copy.
- Prefix increment generally requires less memory and copying time.
- Prefix increment can improve performance when copying is expensive.

Q6 – The Rules of Operator Overloading 7 Marks

Bloom's Level: K5 – Evaluate

Problem Context – A Simple Text Class for an SMS Gateway

An SMS gateway uses its own Text class (holding a dynamically allocated character array) so that it can control memory carefully. It needs assignment, concatenation with +, comparison and printing.

Implementation Tasks

You must:

• Implement Text with a constructor, copy constructor, copy assignment and destructor • Overload + for concatenation, == and != for comparison and << for printing • Prevent self-assignment problems

• Test long messages and empty messages

Design & Analysis

You must:

• List the operators that cannot be overloaded (::, ., .*, ?:, sizeof) and explain why

• Explain the rules: precedence and arity cannot change, at least one operand must be a user-defined type

• Explain why operator= must be a member function

• Explain when an overloaded operator should be a friend

Experimental Investigation

You must:

• Count the copies made when a + b + c is evaluated

• Run the program under the address sanitizer and fix any reported error •

Compare your Text class with std::string for 100,000 concatenations • Test

assigning a Text to itself

Conclusion

Your Text class works, but std::string exists. What did writing Text teach you, and when would you still write such a class?

Alternative Concept and Complexity Analysis

If the Text class were replaced by std::string, or if + were implemented with += on a copy (Text operator+(Text a, const Text& b) { a += b; return a; }), what would the solution be? Analyse the time to evaluate a + b + c for strings of lengths n1, n2, n3 (the characters copied) and the temporary space created in each version.

ANSWER:

1. C++ Program

This program implements a custom Text class using dynamic memory allocation. It overloads assignment (=), concatenation (+), comparison (==, !=) and output (<<) operators.

```
#include <iostream>

#include <cstring>

using namespace std;

class Text {

private:

    char* str;

public:

    // Constructor

    Text(const char* s = "") {

        str = new char[strlen(s) + 1];

        strcpy(str, s);

    }

  // Copy constructor
```

```
    Text(const Text& t) {
        str = new char[strlen(t.str) + 1];
        strcpy(str, t.str);
    }
  // Copy assignment
    Text& operator=(const Text& t) {
        if (this != &t) {
            char* temp = new char[strlen(t.str) + 1];
            strcpy(temp, t.str);
            delete[] str;
            str = temp;
        }
        return *this;
    }
 // Concatenation
    Text operator+(const Text& t) const {
        char* temp = new char[strlen(str) + strlen(t.str) + 1];
strcpy(temp, str);
        strcat(temp, t.str);
Text result(temp);
        delete[] temp;
 return result;
    }
   // Equality
    bool operator==(const Text& t) const {
        return strcmp(str, t.str) == 0;
    }
```

```
// Not equal
    bool operator!=(const Text& t) const {
        return !(*this == t);
    }
// Friend output operator
    friend ostream& operator<<(ostream& out, const Text& t) {
        out << t.str;
        return out;
    }
// Destructor
    ~Text() {
        delete[] str;
    }
};
int main() {
    Text a("Hello");
    Text b(" World");
    Text c("");
Text d = a + b;
cout << "Text 1: " << a << endl;
    cout << "Text 2: " << b << endl;
    cout << "Concatenation: " << d << endl;
if (a == b)
        cout << "Equal" << endl;
    else
        cout << "Not Equal" << endl;
Text e;
```

```
    e = d;
cout << "Assigned Text: " << e << endl;
// Self-assignment
    e = e;
    cout << "After self-assignment: " << e << endl;
// Empty text
    cout << "Empty Text: [" << c << "]" << endl;
// Long message
    Text longMsg("This is a long SMS gateway message.");
    cout << "Long Message: " << longMsg << endl;
return 0;
}
```

2. Design and Analysis

A. Operators that cannot be overloaded

Operator	Meaning	Reason
::	Scope resolution	Controls scope access
.	Member access	Accesses class members
.*	Pointer-to-member access	Built-in member access
?:	Conditional	Controls conditional expressions
sizeof	Size operator	Determines object or type size

These operators have fixed meanings in C++ and cannot be overloaded.

B. Rules of operator overloading

- Operator precedence cannot be changed.
- Operator associativity cannot be changed.
- Operator arity (number of operands) cannot be changed.
- At least one operand must be a user-defined type.
- New operators cannot be created.

- Built-in operators retain their original meaning for built-in types.

C. Why must operator= be a member function?

The assignment operator = must be overloaded as a non-static member function because it modifies the left-hand object.

It also allows direct access to the private data members of the class.

Self-assignment is prevented using:

if (this != &t)

This avoids unnecessary memory deletion and copying when an object is assigned to itself.

D. When should an operator be a friend?

An operator should be a friend when it needs access to private members but the left-hand operand is not an object of that class.

Example:

friend ostream& operator<<(ostream& out, const Text& t);

The output operator uses cout as its left-hand operand, so it cannot be implemented as a normal Text member operator.

3. Experimental Investigation

A. Count the copies in a + b + c

Consider:

Text a("Hello");

Text b("World");

Text c("Students");

Text result = a + b + c;

The expression is evaluated from left to right because + is left-associative:

Text result = (a + b) + c;

Copy operation explanation:

- First, a + b creates a new concatenated Text object.
- The intermediate result is concatenated with c.
- A final Text object stores the result.

The number of copy constructor calls depends on compiler copy elision. In modern C++, some temporary copies can be eliminated automatically.

To count copies, add this inside the Text class:

```
static int copyCount;
```

Inside the copy constructor:

```
Text(const Text& t) {
    ++copyCount;
    str = new char[strlen(t.str) + 1];
    strcpy(str, t.str);
}
```

Outside the class:

```
int Text::copyCount = 0;
```

After concatenation:

```
cout << "Copies: " << Text::copyCount << endl;
```

This displays the actual copy constructor calls.

B. Run using Address Sanitizer

Address Sanitizer helps identify memory errors such as memory leaks, invalid memory access and double deletion.

Compile using:

```
g++ program.cpp -fsanitize=address -g -o test
./test
```

Expected result:

- No memory errors.
- No double deletion.
- No invalid memory access.
- Dynamically allocated memory is released by the destructor.

C. Compare Text with std::string

The following program compares the time required for 100,000 concatenations.

```
#include <iostream>
#include <string>
#include <chrono>
using namespace std;
using namespace chrono;
int main() {
    const int N = 100000;
  string s = "";
    auto start = steady_clock::now();
for (int i = 0; i < N; i++)
        s = s + "A";
auto end = steady_clock::now();
cout << "std::string time: "
         << duration<double, milli>(end-start).count()
         << " ms\n";
cout << "Final length: " << s.length() << endl;
}
```

For the custom Text class, perform the same operation using:

```
Text result("");
for (int i = 0; i < 100000; i++)
    result = result + Text("A");
```

Measure both using chrono and compare the results**.**

Feature	Custom Text	std::string
Memory management	Manual	Automatic
Concatenation	Overloaded +	Built-in class support
Memory safety	Programmer responsible	Standard library manages memory
Performance	Depends on implementation	Highly optimized

Ease of use	More code	Simple to use

Repeated concatenation using + can be slow for both implementations because the growing string is repeatedly copied. Actual execution times depend on compiler and system.

D. Test self-assignment

Program:

```
Text a("Hello");

a = a;

cout << a;
```

Expected output:

Hello

The condition if (this != &t) prevents the object from deleting its own data before copying it.

4. Alternative Concept and Complexity Analysis

There are three possible approaches.

Method 1: Using custom Text class

The programmer manages dynamic memory using new and delete[]. Operator overloading provides convenient concatenation and comparison.

Method 2: Using std::string

```
string a = "Hello";

string b = "World";

string c = a + b;

cout << c;
```

std::string automatically manages memory and provides built-in concatenation and comparison.

Method 3: Using operator+ with +=

```
Text operator+(Text a, const Text& b) {
    a += b;
    return a;
}
```

Here, the left operand is passed by value. The += operator appends the second string to the copied first string.

This avoids allocating a separate intermediate character buffer inside operator+, although copying the left operand is still required.

A. Time complexity of a + b + c

Let:

- n1 = length of string a
- n2 = length of string b
- n3 = length of string c

For:

result = a + b + c;

The operation happens in two stages:

Operation	Characters copied
a + b	n1 + n2
(a + b) + c	n1 + n2 + n3
Total	2n1 + 2n2 + n3

Therefore, total time complexity is:

\[O(n_1+n_2+n_3) \]

This assumes each concatenation copies the required characters once.

B. Temporary memory usage

- First concatenation creates a temporary string of length n1+n2.
- Second concatenation creates the final string of length n1+n2+n3.
- The intermediate temporary must remain available until the second concatenation finishes.

Therefore, temporary storage is proportional to the total string length:

\[O(n_1+n_2+n_3) \]

C. Comparison of approaches

Approach	Time complexity	Memory management

Custom Text with +	O(n1+n2+n3)	Manual
std::string with +	O(n1+n2+n3)	Automatic
Copy + +=	O(n1+n2+n3)	Manual

All three approaches have linear complexity for this two-stage concatenation, assuming ordinary character copying.

5. Conclusion

Writing a custom Text class teaches us:

- How dynamic memory allocation works using new and delete[].
- How copy constructors and assignment operators work.
- How operator overloading makes objects easier to use.
- How to prevent self-assignment and memory errors.
- How to manage temporary objects and improve performance.

When would we write a custom class?

A custom class is useful when we need special memory management, custom text processing, memory control or specific application requirements.

Final conclusion: std::string is preferred for normal applications because it is safer and easier to use. A custom Text class is useful when we need to learn or control memory management and operator behaviour.

Important additional point for the exam: The alternative operator+ using += requires an operator+= implementation in the Text class. It can be written as:

```
Text& operator+=(const Text& t) {
   char* temp = new char[strlen(str) + strlen(t.str) + 1];
strcpy(temp, str);
   strcat(temp, t.str);
delete[] str;
   str = temp;
return *this;
}
```

This allows concatenation without creating an additional intermediate character buffer inside

operator+.

Q7 – Design a Complex-Number Class from First Principles 10 Marks

Bloom's Level: K5 – Evaluate

Problem Context – AC Circuit Impedance Calculator

An electrical engineering laboratory needs a calculator for AC circuits. Impedances are complex numbers: resistors give R + 0j, inductors 0 + jωL and capacitors 0 − j/(ωC). Series impedances add; parallel impedances combine as (Z1 × Z2) / (Z1 + Z2). The calculator must print the impedance in rectangular form and its magnitude, and must handle division by zero safely.

You are not given the design. You must develop it from the

requirements. Stage 1 – Interpret the Problem | 2 Marks

• Identify the data and the operations needed

• Identify which constructors are needed (default, from real part, from real and imaginary parts)

• State the requirements as measurable statements

Stage 2 – Design | 3 Marks

• Design the Complex class with constructors and overloaded +, -, *, /, == and << •

Decide which operators are members and which are friends

• Present the class design and the formula used for each operator

Stage 3 – Implement and Verify | 3 Marks

• Implement the class and compute the impedance of a series and a parallel RLC circuit
• Verify the results by hand calculation

• Test division by a zero impedance

Stage 4 – Defend the Design | 2 Marks

• Compare your class with std::complex<double>

• Explain why each constructor and operator is needed

• Identify one remaining limitation

Alternative Concept and Complexity Analysis

If std::complex<double> were used instead of your own Complex class, what would the solution be? Analyse the time per operation (O(1) in both) and the space (two doubles in both), and compare accuracy and the work needed to handle division by zero.

Submission Requirements

GitHub repository link (mandatory) containing the executed code for every task, including every completed “Complete the Code” program, with input files, output screenshots and a README.md.

• Your report containing the required evidence for Q1–Q7

• C++ source code for all questions

• Predictions, outputs, memory diagrams, sanitizer reports and timings

Assessment Verification

Students may be asked to explain, execute or modify any part of their design, code, tests or results during a brief verification discussion/viva. The student should be able to explain and defend the work submitted as their own.

ANSWER:

Design a Complex-Number Class from First Principles

Problem: AC Circuit Impedance Calculator

Stage 1 – Interpret the Problem

1. Identify the data and operations needed

In an AC circuit, impedance is represented using a complex number:

\[Z = a + jb \]

Where:

- a = Real part (Resistance)
- b = Imaginary part (Reactance)
- j = Imaginary unit

Required data:

- Real part (real)
- Imaginary part (imag)

Required operations:

1. Addition (+) – Series impedance.

2. Subtraction (-) – Difference between impedances.
3. Multiplication (*) – Product of complex numbers.
4. Division (/) – Parallel impedance calculation.
5. Equality (==) – Compare two impedances.
6. Output (<<) – Display impedance.

2. Required constructors

Three constructors are needed:

Constructor	Purpose
Default constructor	Initializes both parts to zero
One-argument constructor	Initializes only the real part
Two-argument constructor	Initializes real and imaginary parts

Example:

```
Complex a;        // 0 + 0j
Complex b(5);     // 5 + 0j
Complex c(3, 4);  // 3 + 4j
```

3. Measurable requirements

- The class must store two double values.
- All arithmetic operators must produce correct results.
- Division by zero must be handled safely.
- Impedance must be displayed in rectangular form.
- Magnitude must be calculated correctly.
- Series and parallel circuit calculations must be verified.

Stage 2 – Design

1. Complex class design

Class: Complex

Private Data:

```
double real;
```

double imag;

Public:

Default Constructor

One-Argument Constructor

Two-Argument Constructor

operator+()

operator-()

operator*()

operator/()

operator==()

magnitude()

operator<<()

2. Formulas used

Let two complex numbers be:

$$ Z_1=a+jb,\quad Z_2=c+jd $$

Operator	Formula
Addition	(a+c) + j(b+d)
Subtraction	(a-c) + j(b-d)
Multiplication	(ac-bd) + j(ad+bc)
Division	$[(ac+bd)+j(bc-ad)]/(c^2+d^2)$
Equality	Both real and imaginary parts are equal

Magnitude formula:

$$ |Z|=\sqrt{a^2+b^2} $$

3. Member and friend operators

- +, -, *, /, == are implemented as member functions.
- << is implemented as a friend function because cout is the left-hand operand.
- The friend function can access the private real and imaginary values.

Stage 3 – Implement and Verify (3 Marks)

1. Complete C++ program

```
#include <iostream>
#include <cmath>
#include <stdexcept>
#include <iomanip>
using namespace std;
class Complex {
private:
    double real, imag;
public:
    // Default constructor
    Complex() : real(0), imag(0) {}
// Constructor with real part
    Complex(double r) : real(r), imag(0) {}
// Constructor with real and imaginary parts
    Complex(double r, double i) : real(r), imag(i) {}
// Addition
    Complex operator+(const Complex& c) const {
        return Complex(real + c.real, imag + c.imag);
    }
// Subtraction
    Complex operator-(const Complex& c) const {
        return Complex(real - c.real, imag - c.imag);
    }
// Multiplication
    Complex operator*(const Complex& c) const {
```

```
        return Complex(
            real * c.real - imag * c.imag,
            real * c.imag + imag * c.real
        );
    }
// Division
    Complex operator/(const Complex& c) const {
        double denominator =
            c.real * c.real + c.imag * c.imag;
if (denominator == 0.0)
            throw domain_error("Division by zero impedance");
return Complex(
            (real * c.real + imag * c.imag) / denominator,
            (imag * c.real - real * c.imag) / denominator
        );
    }
// Equality
    bool operator==(const Complex& c) const {
        const double EPS = 1e-9;
        return fabs(real - c.real) < EPS &&
               fabs(imag - c.imag) < EPS;
    }
// Magnitude
    double magnitude() const {
        return hypot(real, imag);
    }
// Friend output operator
```

```
    friend ostream& operator<<(ostream& out, const Complex& c) {
        out << c.real;

        if (c.imag >= 0)
            out << " + j" << c.imag;
        else
            out << " - j" << fabs(c.imag);
return out;
    }
};

// Calculate parallel impedance
Complex parallel(Complex a, Complex b) {
    return (a * b) / (a + b);
}
int main() {
    try {
        const double PI = 3.141592653589793;
        const double f = 50.0;
        const double R = 10.0;
        const double L = 0.1;
        const double C = 100e-6;
double omega = 2 * PI * f;
double XL = omega * L;
        double XC = 1 / (omega * C);
Complex resistor(R);
        Complex inductor(0, XL);
```

```
        Complex capacitor(0, -XC);
// Series RLC impedance
        Complex series = resistor + inductor + capacitor;
cout << fixed << setprecision(4);
cout << "Series Impedance: " << series << endl;
        cout << "Series Magnitude: "
             << series.magnitude() << endl;
// Parallel RLC impedance
        Complex p1 = parallel(resistor, inductor);
        Complex p2 = parallel(p1, capacitor);
cout << "\nParallel Impedance: " << p2 << endl;
        cout << "Parallel Magnitude: "
             << p2.magnitude() << endl;
// Division by zero test
        Complex zero(0, 0);
cout << "\nDivision by zero test: ";
        Complex result = resistor / zero;
        cout << result << endl;
    }
    catch (const domain_error& e) {
        cout << e.what() << endl;
    }
return 0;
}
```

2. Verify results by hand calculation

Assume:

- Resistance R = 10 Ω

- Inductance L = 0.1 H
- Capacitance C = 100 μF
- Frequency f = 50 Hz

Step 1: Calculate angular frequency

$$\omega=2\pi f$$

$$\omega=2\times3.14159\times50 =314.159$$

Step 2: Calculate inductive reactance

$$X_L=\omega L$$

$$X_L=314.159\times0.1=31.416\ \Omega$$

Step 3: Calculate capacitive reactance

$$X_C=\frac{1}{\omega C}$$

$$X_C=\frac{1}{314.159\times100\times10^{-6}}$$

$$X_C=31.831\ \Omega$$

A. Series RLC circuit

Formula:

$$Z=Z_R+Z_L+Z_C$$

$$Z=10+j31.416-j31.831$$

Series result

$$\boxed{Z=10-j0.415\ \Omega}$$

Magnitude:

$$|Z|=\sqrt{10^2+(-0.415)^2}$$

$$\boxed{|Z|=10.0086\ \Omega}$$

B. Parallel RLC circuit

For three parallel impedances:

$$\frac{1}{Z}=\frac{1}{Z_R}+\frac{1}{Z_L}+\frac{1}{Z_C}$$

Substituting the values:

$$\frac{1}{Z}=0.1-j0.031831+j0.031416$$

$$\frac{1}{Z}=0.1-j0.000415$$

Taking the reciprocal:

$$\boxed{Z\approx9.9998+j0.0415\ \Omega}$$

Magnitude:

$$|Z|=\sqrt{9.9998^2+0.0415^2}$$

$$\boxed{|Z|\approx9.9999\ \Omega}$$

3. Expected program output

Series Impedance: 10.0000 - j0.4151

Series Magnitude: 10.0086

Parallel Impedance: 9.9998 + j0.0415

Parallel Magnitude: 9.9999

Division by zero test: Division by zero impedance

Verification: The calculated series and parallel values match the expected program results, allowing for rounding.